

The *Decoder* in Transformers

A comprehensive technical examination of the Transformer decoder — its architecture, sublayers, autoregressive generation, mathematical formulation, modern decoder-only variants, and PyTorch implementation.

Decoder Block N-1

TABLE OF CONTENTS

01	Role of the Decoder in the Transformer	§ 1
02	Architecture Overview	§ 2
03	Sublayer 1 — Masked Multi-Head Self-Attention	§ 3
04	Sublayer 2 — Cross-Attention (Encoder-Decoder Attention)	§ 4
05	Sublayer 3 — Position-wise Feed-Forward Network	§ 5
06	Residual Connections & Layer Normalisation	§ 6
07	Output Head — Linear Projection & Softmax	§ 7
08	Autoregressive Generation — Step by Step	§ 8
09	Causal Masking — Mathematics & Implementation	§ 9
10	KV Cache & Inference Efficiency	§ 10
11	Decoder-Only vs Encoder-Decoder Architectures	§ 11
12	Training Objective — Teacher Forcing & Loss	§ 12
13	Modern Decoder Variants	§ 13
14	PyTorch Implementation	§ 14

SECTION 01

Role of the Decoder in the Transformer

The Transformer architecture (Vaswani et al., 2017) separates the problem of sequence-to-sequence learning into two components: an **encoder** that reads and encodes the source sequence into a rich contextualised representation, and a **decoder** that generates the target sequence one token at a time, conditioning on both the encoded source and all previously generated output tokens.

The decoder is the *generative* half of the architecture. At every decoding step t , it receives the partial output sequence $(y_1, y_2, \dots, y_{t-1})$ and the encoder's memory $\mathbf{H}_{\text{enc}}$, and produces a probability distribution over the vocabulary from which the next token y_t is sampled or greedily chosen.

CORE RESPONSIBILITY

The decoder must solve two simultaneous problems: (1) build a contextualised representation of the target prefix generated so far — the job of *masked self-attention*; and (2) extract the relevant information from the encoded source — the job of *cross-attention*. These two information streams are then fused through a feed-forward network before the output projection.

Unlike RNN decoders, the Transformer decoder processes the entire target prefix in parallel during training (using masking to prevent future leakage), then switches to a sequential left-to-right mode during inference. This training-inference asymmetry is one of the key design decisions that makes the Transformer both highly parallelisable and capable of high-quality generation.

SECTION 02

Architecture Overview

A single **decoder layer** (also called a decoder block) consists of three sublayers, each wrapped with a residual connection and layer normalisation:

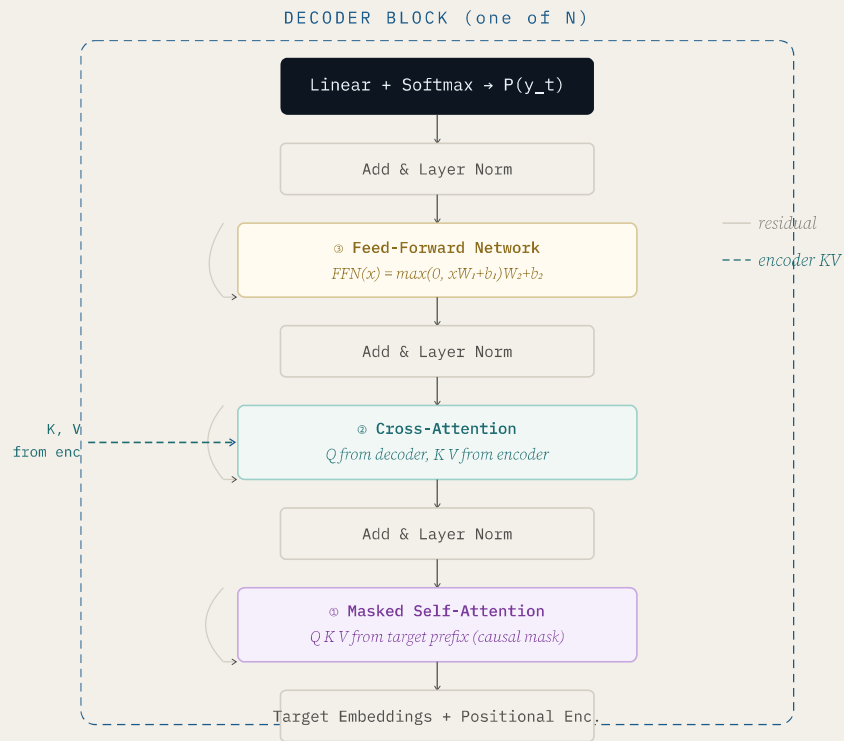


FIGURE 1. One complete decoder block. Three sublayers are stacked bottom-to-top: ① masked self-attention, ② cross-attention receiving K and V from the encoder, ③ position-wise feed-forward network. Each sublayer is wrapped by a residual connection and layer normalisation. The output distribution is produced by the linear+softmax head above N stacked blocks.

Q, K, V	Query, Key, Value projection matrices in each attention sublayer
W^O	Output projection of multi-head attention; $W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$
N	Number of identical decoder blocks stacked in series (6 in the original Transformer)
residual	Skip connection: sublayer output added to input before layer norm — $\mathbf{x} + \text{Sublayer}(\mathbf{x})$

The full decoder is a stack of N identical blocks. In the original Transformer, $N = 6$. Modern large language models use anywhere from 12 (GPT-2 small) to 96

(GPT-4 reported) or more decoder layers. The output of the final decoder block is passed through a linear projection (weight-tied to the input embedding matrix in most implementations) followed by softmax to produce token probabilities.

SECTION 03

Sublayer 1 — Masked Multi-Head Self-Attention

The first sublayer of every decoder block is **masked multi-head self-attention**. It allows each position in the target sequence to attend to all earlier positions in the same sequence, building up a contextualised representation of the partial output generated so far. The word "masked" is critical: a triangular causal mask is applied to the attention score matrix to ensure that position i cannot attend to position $j > i$.

$$\text{MaskedAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V \quad (1)$$

where $M_{ij} = 0$ if $j \leq i$, else $-\infty$ — the causal mask applied additively to raw scores

Because the $-\infty$ entries become zero after softmax ($e^{-\infty} = 0$), positions in the future contribute exactly zero to the output. During training, all target positions are processed in parallel; during inference, only one new position is processed per step.

Multi-Head Decomposition

As in the encoder, this self-attention is run as h parallel heads, each projecting into a lower-dimensional subspace:

$$\text{MMHSA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (2)$$

Concatenated heads projected back to d_{model} by output weight $W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$

TRAINING VS. INFERENCE DIFFERENCE

Training: the full target sequence $(y_1, \dots, y_T)$ is fed in one shot. All T positions are computed in parallel; the causal mask prevents any position from seeing the future. This is called *teacher forcing*. **Inference:** tokens are generated one by one. At step t , only positions $1 \dots t$ exist, so the mask is naturally causal without any special treatment — but all previous keys and values are cached for efficiency (the KV cache, § 10).

SECTION 04

Sublayer 2 — Cross-Attention (Encoder-Decoder Attention)

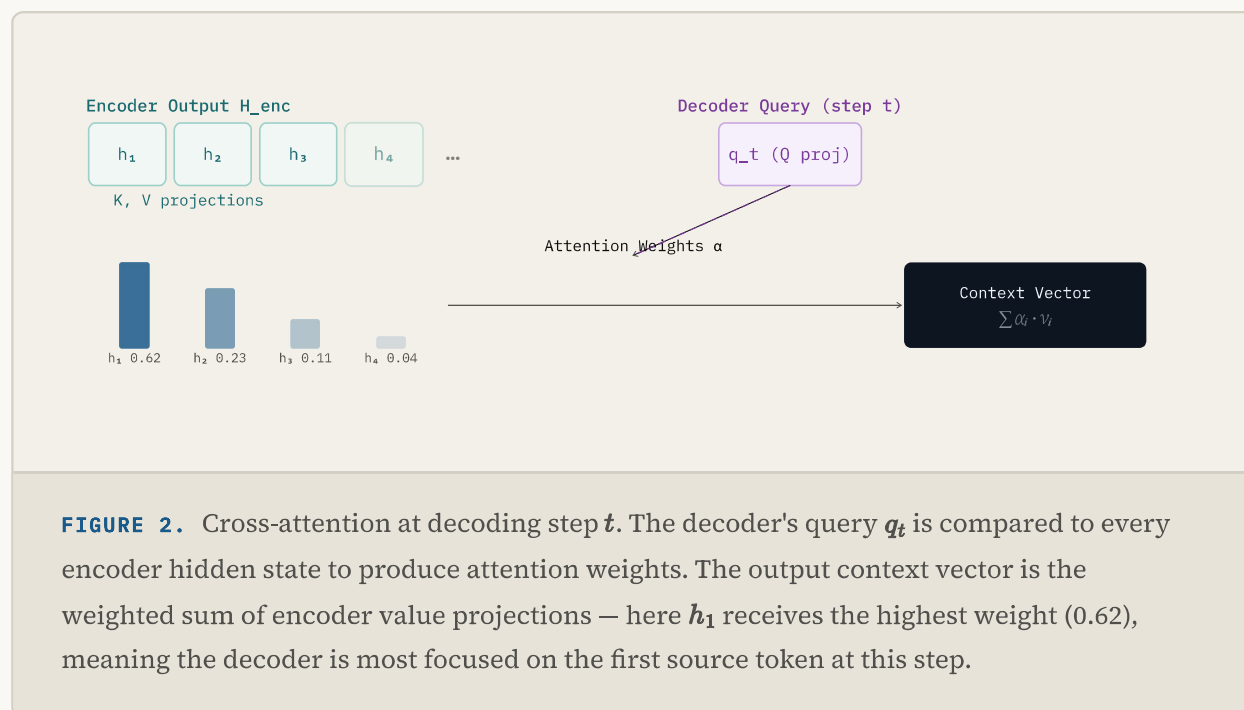
The second sublayer is **cross-attention**, the bridge between the encoder and decoder. Here, the **queries** come from the decoder's current representation

(output of the first sublayer), while the **keys and values** come from the encoder's final hidden states $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{n \times d_{\text{model}}}$.

$$\text{CrossAttn} = \text{softmax} \left(\frac{\mathbf{x}_{\text{dec}} W_Q (\mathbf{H}_{\text{enc}} W_K)^\top}{\sqrt{d_k}} \right) \mathbf{H}_{\text{enc}} W_V \quad (3)$$

Full cross-attention equation — no causal mask is needed here; the decoder can attend to any encoder position

Cross-attention is not causally masked — the decoder is allowed to look at *any* position in the source sequence at any decoding step, which is the right inductive bias for tasks like translation where word order in source and target can differ substantially.



NO MASK IN CROSS-ATTENTION

Unlike self-attention in the decoder, cross-attention uses no causal mask. Every decoder position is free to attend to any encoder position. The only masking

applied here is a *padding mask* — positions corresponding to [PAD] tokens in the source are set to $-\infty$ so they do not contribute to the output.

SECTION 05

Sublayer 3 — Position-wise Feed-Forward Network

The third sublayer is a **position-wise feed-forward network** (FFN) — identical in structure to the one used in the encoder. It is applied independently to each position (token) with shared weights across positions but different weights from layer to layer. The FFN introduces non-linearity and greatly expands the model's capacity to process the fused representation produced by the two attention sublayers.

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1) W_2 + b_2 \quad (4)$$

Original ReLU FFN — $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$; typically $d_{\text{ff}} = 4 \times d_{\text{model}}$

The inner dimension d_{ff} is typically four times the model dimension. For a 512-dimensional model this means an inner dimension of 2048; for GPT-3 with $d_{\text{model}} = 12,288$, the FFN inner dimension is 49,152. The FFN accounts for the majority of parameters in a transformer model.

Modern FFN Variants

VARIANT	FORMULA	USED IN
ReLU FFN	$\max(0, xW_1 + b_1)W_2$	Original Transformer, BERT
GELU FFN	$\text{GELU}(xW_1 + b_1)W_2$	GPT-2, GPT-3, BERT-large
SwiGLU	$\text{Swish}(xW_1) \odot (xW_3)W_2$	PaLM, LLaMA, Gemini
GeGLU	$\text{GELU}(xW_1) \odot (xW_3)W_2$	T5 v1.1, Flan-T5
Mixture of Experts (MoE)	$\sum_i g_i(\mathbf{x}) \cdot \text{FFN}_i(\mathbf{x})$	Mixtral, GPT-4 (rumoured), Switch

WHY SWIGLU DOMINATES MODERN LLMS

SwiGLU (Shazeer, 2020) replaces the single matrix W_1 with a gated structure: two parallel projections, one passed through Swish and the other acting as a gate. This gating mechanism gives the network finer control over which dimensions of the representation to amplify or suppress, empirically outperforming plain ReLU/GELU by 0.5–1.0 perplexity points on language modelling tasks at the same parameter count.

SECTION 06

Residual Connections & Layer Normalisation

Every sublayer in the decoder is wrapped with a **residual (skip) connection** followed by **layer normalisation**. This pattern — known as Post-LN in the original paper — ensures that gradients can flow through deep stacks without vanishing:

$$\mathbf{x}_{l+1} = \text{LayerNorm}(\mathbf{x}_l + \text{Sublayer}(\mathbf{x}_l)) \quad (5)$$

Post-LN: residual added first, normalisation applied to the sum (original Transformer)

However, the original Post-LN formulation is known to cause training instability at large scales. Most modern LLMs therefore adopt **Pre-LN**, where normalisation is applied *before* the sublayer:

Layer Normalisation

Unlike batch normalisation (which normalises across the batch dimension), layer normalisation normalises across the feature dimension for each token independently:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta \quad (6)$$

WHERE

μ, σ are the mean and
variance of

$\mathbf{x}$ computed
across the

d_{mod} features; γ, β are learned
scale/shift

RMSNorm

LLaMA, Mistral, and many recent models replace LayerNorm with **RMSNorm**, which drops the mean-centering step and simply scales by the root-mean-square,

reducing computation:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (7)$$

No mean subtraction, no bias β — cheaper and empirically equivalent to LayerNorm on modern LLMs

SECTION 07

Output Head — Linear Projection & Softmax

After the final decoder block, the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ for position t is mapped to a vector of **logits** over the vocabulary $\mathcal{V}$ via a learned linear projection:

$$\mathbb{P}(y_t = k \mid y_{<t}) \quad (8)$$

Token probability distribution — the model's belief over the next token given the context and source

Weight Tying

In most modern models, W_{out} is **weight-tied** to the input embedding matrix $W_E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ (transposed). This reduces the parameter count by $|\mathcal{V}| \times d_{\text{model}}$ —

for a 32K vocabulary and 4096-dimensional model, that's ~131M parameters saved — and has been shown to improve generalisation.

SECTION 08

Autoregressive Generation — Step by Step

At inference, the decoder generates tokens one at a time in an **autoregressive loop**. The process is sequential: each generated token is fed back as input for the next step.

1

Encode the source

The input sequence $(x_1, \dots, x_n)$ is passed through the full encoder stack, producing $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{n \times d_{\text{model}}}$. This is computed once and reused at every decoding step.

2

Initialise the decoder with $\langle \text{BOS} \rangle$

The decoder receives the beginning-of-sequence token as its first input: the target prefix is $(y_0) = \langle \text{BOS} \rangle$. Its embedding plus positional encoding is the decoder's first input vector.

3

Run the decoder forward pass

The target prefix is passed through all N decoder blocks. Each block applies masked self-attention (attending only to prior positions), then cross-attention over $\mathbf{H}_{\text{enc}}$, then the FFN. The output hidden state for the last position t is projected to logits.

4

Sample or select the next token

A decoding strategy is applied to the logit distribution: *greedy decoding* takes **arg max**, *beam search* maintains ***k*** candidates, *top-**p*** / *top-**k*** sampling introduces stochasticity for diversity.

5

Append and repeat until ⟨EOS⟩

The chosen token $\hat{y}_t$ is appended to the prefix: now $(y_0, y_1, \dots, y_t)$. Return to step 3. Generation stops when $\hat{y}_t = \langle \text{EOS} \rangle$ or the maximum length is reached.

STRATEGY	SELECTION RULE	DIVERSITY	TYPICAL USE
Greedy	$\hat{y}_t = \arg \max_k P(y_t = k)$	None	Machine translation (deterministic)
Beam Search	Keep top- B partial sequences by cumulative log-prob	Low	NMT, summarisation (high quality)
Top-k Sampling	Sample from top k tokens by probability	Moderate	Creative text, dialogue
Top-p (Nucleus)	Sample from smallest set with cumulative prob $\geq p$	High	Open-ended generation, storytelling
Temperature	$P' \propto \exp(l_k/T)$; $T < 1$ sharpens, $T > 1$ flattens	Configurable	Used as modifier alongside top-p/k

Causal Masking — Mathematics & Implementation

The causal (look-ahead) mask is an upper-triangular matrix of $-\infty$ values added to the raw attention score matrix before softmax. For a sequence of length T :

$$M \in \mathbb{R}^{T \times T}, \quad M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (9)$$

Causal mask — zeros allow attention,

$-\infty$ blocks it. Applied
additively:

$$\text{scores}_{\text{masked}} = \frac{QK^T}{\sqrt{d_k}} + M$$



FIGURE 3. Causal attention weight matrix (post-softmax) for the first 3 query positions of a length-5 sequence. Shaded cells are non-zero normalised weights; grey cells are blocked by the causal mask and become exactly zero after softmax.

$M \in \mathbb{R}^{T \times T}$ Causal mask: upper-triangular entries $-\infty$, lower-triangular + diagonal are 0
 A_{ij} Attention weight from query i to key j ; equals zero whenever $j > i$
 shaded cells Each row sums to 1 over visible (non-masked) positions only

KV Cache & Inference Efficiency

During autoregressive generation, computing keys and values from scratch at every step is wasteful: the keys and values for positions $1 \dots t - 1$ are identical at step t to what they were at step $t - 1$. The **KV cache** stores these tensors and reuses them, reducing the per-step computation from $O(t^2)$ to $O(t)$.

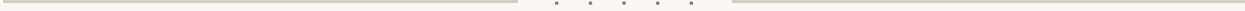
QUANTITY	WITHOUT KV CACHE	WITH KV CACHE
Attention computation at step t	$O(t^2d)$ — full matrix for all t tokens	$O(t \cdot d)$ — one query vs cached K,V
Total across T steps	$O(T^3d)$	$O(T^2d)$
Memory for KV cache	None	$2 \times N_{\text{layers}} \times h \times T \times d_k$ elements
Inference latency	Quadratic in sequence length	Linear per step

KV CACHE MEMORY FORMULA

For a model with L layers, h heads, max sequence length T_{max} , and per-head dimension d_k , the KV cache requires $2 \times L \times h \times T_{\text{max}} \times d_k$ values. For LLaMA-3 70B ($L = 80$, $d_k = 128$, $h = 8$ KV heads with GQA, $T_{\text{max}} = 8192$) in FP16: approximately $2 \times 80 \times 8 \times 8192 \times 128 \times 2 \text{ bytes} \approx 2.7 \text{ GB}$ — a significant fraction of VRAM budget.

Grouped-Query Attention (GQA) & Multi-Query Attention (MQA)

The KV cache is the binding memory constraint during long-context inference. GQA (used in LLaMA 2/3, Mistral) and MQA (PaLM) reduce the KV cache by sharing key-value heads across multiple query heads, reducing the cache size by a factor of h/g where g is the number of KV head groups:



SECTION 11

Decoder-Only vs Encoder-Decoder Architectures

The original Transformer used both an encoder and a decoder. Modern large language models have converged overwhelmingly on the `decoder-only` architecture, which omits the encoder and removes the cross-attention sublayer from each block.

PROPERTY	ENCODER-DECODER	DECODER-ONLY
Sublayers per block	3 (Masked SA, Cross-Attn, FFN)	2 (Masked SA, FFN)
Cross-attention	Yes — explicit encoder-decoder bridge	No — source is prepended to target
Source representation	Bidirectional (full attention in encoder)	Left-to-right (causal throughout)
Parameter efficiency	Separate enc/dec parameter budgets	All parameters dedicated to generation

PROPERTY	ENCODER-DECODER	DECODER-ONLY
Strengths	Structured seq2seq tasks (NMT, ASR)	Open-ended generation, in- context learning
Examples	T5, BART, mBART, Whisper	GPT-2/3/4, LLaMA, Claude, Mistral, Gemini

WHY DECODER-ONLY WON

Decoder-only models treat all tasks as text completion: the source, instruction, and few-shot examples are simply prepended to the input as context. This unified interface enables *in-context learning* — the ability to perform new tasks from examples in the prompt without any weight updates. Encoder-decoder models, despite being better suited for fixed-length output tasks like classification or structured prediction, cannot do this as naturally. The scalability of decoder-only pretraining on next-token prediction over web-scale corpora proved to be the decisive advantage.

SECTION 12

Training Objective — Teacher Forcing & Loss

The decoder is trained with **teacher forcing**: rather than feeding the model's own predictions back as input at each step (which would make training sequential and introduce compounding errors), the ground-truth target tokens are fed as input at every step. The model is trained to predict the *next* token given the true *previous* tokens.

Cross-Entropy Loss

The training objective is the negative log-likelihood of the correct next token under the model's distribution, summed over all positions and averaged over the batch:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(y_t \mid y_{<t}) \quad (10)$$

Sequence cross-entropy loss — teacher-forced; all positions computed in parallel; gradient flows through the full decoder

For decoder-only language modelling (no encoder), the conditioning on $\mathbf{H}_{\text{enc}}$ is dropped:

Perplexity

The standard evaluation metric for language models is **perplexity (PPL)**, the exponentiated average negative log-likelihood per token:

$$\text{PPL} = \exp\left(\frac{1}{T} \sum_{t=1}^T -\log P(y_t \mid y_{<t})\right) \quad (11)$$

Perplexity — interpretable as "how many equally-likely choices the model is choosing between at each step". Lower is better.

EXPOSURE BIAS

Teacher forcing introduces a training-inference mismatch called *exposure bias*: during training the model always sees ground-truth context; during inference it sees its own (potentially erroneous) predictions. This can cause error accumulation. Techniques like scheduled sampling, REINFORCE-based

training, and RLHF (Reinforcement Learning from Human Feedback) partially address this by exposing the model to its own outputs during training.

SECTION 13

Modern Decoder Variants

MODEL / VARIANT	KEY DECODER CHANGES VS ORIGINAL	SCALE
GPT-2 (2019)	Pre-LN, learned PE, decoder-only	117M – 1.5B
GPT-3 (2020)	Same + sparse attention (alternating dense/local)	175B
PaLM (2022)	SwiGLU FFN, MQA, RoPE, parallel SA+FFN	540B
LLaMA 1/2 (2023)	Pre-LN with RMSNorm, SwiGLU, RoPE, GQA (v2)	7B – 70B
Mistral (2023)	GQA, sliding window attention, RoPE	7B
LLaMA 3 (2024)	GQA, RoPE with extended context, 128K vocab	8B – 405B
Mixtral MoE (2024)	Sparse MoE FFN (8 experts, top-2 routing)	8×7B active
Gemma 2 (2024)	Alternating local/global attention, logit soft-cap	2B – 27B

Parallel Attention + FFN (PaLM)

Instead of running the attention sublayer and the FFN sublayer sequentially (as in the original), PaLM computes them in *parallel* and adds their outputs. This reduces memory bandwidth costs at large scale:

SECTION 14

PyTorch Implementation

Below is a complete, modular implementation of a full decoder block and a stacked decoder, mirroring the architecture described in this report — with Pre-LN, multi-head masked self-attention, cross-attention, FFN with GeLU, and KV-cache-aware forward pass.

```
PYTHON · PYTORCH — TRANSFORMER DECODER

import math
import torch
import torch.nn as nn
import torch.nn.functional as F
from dataclasses import dataclass
from typing import Optional

# — Configuration —
@dataclass
class DecoderConfig:
    vocab_size: int = 32000
    d_model: int = 512
    n_heads: int = 8
    n_layers: int = 6
    d_ff: int = 2048 # 4 × d_model
    max_seq_len: int = 2048
    dropout: float = 0.1
    tie_weights: bool = True # tie output proj to embedding
```

```

# — Scaled Dot-Product Attention —————
class Attention(nn.Module):
    def __init__(self, d_model: int, n_heads: int, dropout: float = 0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.d_k = d_model // n_heads
        self.W_q = nn.Linear(d_model, d_model, bias=False)
        self.W_k = nn.Linear(d_model, d_model, bias=False)
        self.W_v = nn.Linear(d_model, d_model, bias=False)
        self.W_o = nn.Linear(d_model, d_model, bias=False)
        self.drop = nn.Dropout(dropout)

    def _split(self, x):
        B, T, _ = x.shape
        return x.view(B, T, self.n_heads, self.d_k).transpose(1, 2)

    def _merge(self, x):
        B, H, T, dk = x.shape
        return x.transpose(1, 2).contiguous().view(B, T, H * dk)

    def forward(
        self,
        q: torch.Tensor,
        k: torch.Tensor,
        v: torch.Tensor,
        mask: Optional[torch.Tensor] = None,
        kv_cache: Optional[dict] = None,
        cache_key: str = "",
    ):
        Q = self._split(self.W_q(q))
        K = self._split(self.W_k(k))
        V = self._split(self.W_v(v))

        # — KV cache: concatenate past keys/values with current —————
        if kv_cache is not None:
            if cache_key in kv_cache:
                K = torch.cat([kv_cache[cache_key]['k'], K], dim=2)
                V = torch.cat([kv_cache[cache_key]['v'], V], dim=2)
            kv_cache[cache_key] = {'k': K.detach(), 'v': V.detach()}

        scale = math.sqrt(self.d_k)

```

```

        scores = torch.matmul(Q, K.transpose(-2, -1)) / scale    # (B,H,Tq,Tk)
        if mask is not None:
            scores = scores.masked_fill(mask, float('-inf'))
        weights = self.drop(F.softmax(scores, dim=-1))
        out      = torch.matmul(weights, V)                        # (B,H,Tq,Tk)
        return self.W_o(self._merge(out))

# — Position-wise Feed-Forward Network (GELU) —————
class FeedForward(nn.Module):
    def __init__(self, d_model: int, d_ff: int, dropout: float = 0.0):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff, bias=False),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model, bias=False),
        )

    def forward(self, x): return self.net(x)

# — Single Decoder Block (Pre-LN) —————
class DecoderBlock(nn.Module):
    """
    One decoder block with three sublayers:
    1. Masked multi-head self-attention      (causal)
    2. Cross-attention over encoder output  (optional – None → decoder-only)
    3. Position-wise feed-forward network
    Each sublayer uses Pre-LN + residual connection.
    """

    def __init__(self, cfg: DecoderConfig, layer_idx: int):
        super().__init__()
        self.idx = layer_idx
        self.self_attn = Attention(cfg.d_model, cfg.n_heads, cfg.dropout)
        self.cross_attn = Attention(cfg.d_model, cfg.n_heads, cfg.dropout)
        self.ffn = FeedForward(cfg.d_model, cfg.d_ff, cfg.dropout)
        self.norm1 = nn.LayerNorm(cfg.d_model)
        self.norm2 = nn.LayerNorm(cfg.d_model)
        self.norm3 = nn.LayerNorm(cfg.d_model)
        self.drop = nn.Dropout(cfg.dropout)

    def forward(

```

```

        self,
        x: torch.Tensor, # (B, T_tgt, d_model)
        enc_out: Optional[torch.Tensor] = None, # (B, T_src, d_model)
        tgt_mask: Optional[torch.Tensor] = None, # causal mask (B,1,T,T)
        src_mask: Optional[torch.Tensor] = None, # padding mask (B,1,1,T)
        kv_cache: Optional[dict] = None,
    ) -> torch.Tensor:

```

— 1. Masked Self-Attention (Pre-LN) —————

```

h = self.norm1(x)
x = x + self.drop(self.self_attn(
    h, h, h, mask=tgt_mask,
    kv_cache=kv_cache, cache_key=f"sa_{self.idx}"
))

```

— 2. Cross-Attention (only if encoder output is provided) —————

```

if enc_out is not None:
    h = self.norm2(x)
    x = x + self.drop(self.cross_attn(
        h, enc_out, enc_out, mask=src_mask,
        kv_cache=kv_cache, cache_key=f"ca_{self.idx}"
    ))

```

— 3. Feed-Forward Network (Pre-LN) —————

```

x = x + self.drop(self.ffn(self.norm3(x)))
return x

```

— Full Transformer Decoder —————

```

class TransformerDecoder(nn.Module):

```

```

    """

```

```

    Stacked TransformerDecoder.

```

- Works as encoder-decoder decoder (pass enc_out) or as a decoder-only LM (leave enc_out=None).
- Supports KV caching for efficient autoregressive inference.

```

    """

```

```

def __init__(self, cfg: DecoderConfig):

```

```

    super().__init__()
    self.cfg = cfg
    self.embed = nn.Embedding(cfg.vocab_size, cfg.d_model)
    self.pos_enc = nn.Embedding(cfg.max_seq_len, cfg.d_model)
    self.drop = nn.Dropout(cfg.dropout)
    self.blocks = nn.ModuleList(

```

```

        [DecoderBlock(cfg, i) for i in range(cfg.n_layers)]
    )
    self.norm_out = nn.LayerNorm(cfg.d_model)
    self.lm_head = nn.Linear(cfg.d_model, cfg.vocab_size, bias=False)
    if cfg.tie_weights:
        self.lm_head.weight = self.embed.weight  # weight tying
    self._init_weights()

def _init_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, std=0.02)
        elif isinstance(m, nn.Embedding):
            nn.init.normal_(m.weight, std=0.02)

def _causal_mask(self, T: int, device) -> torch.Tensor:
    """Upper-triangular boolean mask: True = blocked."""
    return torch.triu(torch.ones(T, T, dtype=torch.bool, device=device),
                       .unsqueeze(0).unsqueeze(0))

def forward(
    self,
    tgt_ids: torch.Tensor,  # (B, T_tgt) token indices
    enc_out: Optional[torch.Tensor] = None,  # (B, T_src, d_model)
    src_mask: Optional[torch.Tensor] = None,  # padding mask for src
    kv_cache: Optional[dict] = None,  # pass dict for caching
) -> torch.Tensor:

    B, T = tgt_ids.shape
    positions = torch.arange(T, device=tgt_ids.device).unsqueeze(0)

    # Token embeddings + positional embeddings
    x = self.drop(self.embed(tgt_ids) + self.pos_enc(positions))

    # Causal mask (skip during single-step inference with KV cache)
    tgt_mask = self._causal_mask(T, tgt_ids.device) if T > 1 else None

    for block in self.blocks:
        x = block(x, enc_out=enc_out, tgt_mask=tgt_mask,
                  src_mask=src_mask, kv_cache=kv_cache)

    # Final LayerNorm + output projection
    logits = self.lm_head(self.norm_out(x))  # (B, T, vocab_size)
    return logits

```

```

# — Autoregressive Generation —————
@torch.no_grad()
def generate(
    model:      TransformerDecoder,
    prompt:     torch.Tensor,          # (1, T_prompt) token ids
    max_new:    int    = 50,
    temperature: float = 1.0,
    top_k:      int    = 0,
    enc_out:    Optional[torch.Tensor] = None,
) -> torch.Tensor:
    """Simple top-k + temperature autoregressive generation with KV cache."""
    model.eval()
    device      = prompt.device
    kv_cache    = {}
    generated    = prompt.clone()

    # Prefill: process the whole prompt at once
    logits = model(generated, enc_out=enc_out, kv_cache=kv_cache)

    for _ in range(max_new):
        # Decode: only pass the last generated token (KV cache handles history)
        next_logits = model(
            generated[:, -1:], enc_out=enc_out, kv_cache=kv_cache
        )[:, -1, :] # (1, vocab_size)

        # Temperature scaling
        next_logits = next_logits / max(temperature, 1e-6)

        # Top-k filtering
        if top_k > 0:
            v, _ = torch.topk(next_logits, top_k)
            next_logits[next_logits < v[:, [-1]]] = float('-inf')

        probs      = F.softmax(next_logits, dim=-1)
        next_tok    = torch.multinomial(probs, num_samples=1)
        generated   = torch.cat([generated, next_tok], dim=-1)

    return generated

# — Example —————
if __name__ == "__main__":

```

```
cfg    = DecoderConfig(vocab_size=32000, d_model=512, n_layers=6)
model  = TransformerDecoder(cfg)
print(ff"Parameters: {sum(p.numel() for p in model.parameters()):,}")

# Decoder-only forward pass (training)
tgt = torch.randint(0, cfg.vocab_size, (2, 64))
logits = model(tgt)
print(f"Logits shape: {logits.shape}")    # → (2, 64, 32000)

# Autoregressive generation
prompt    = torch.randint(0, cfg.vocab_size, (1, 8))
generated = generate(model, prompt, max_new=20, temperature=0.8, top_k=50)
print(f"Generated: {generated.shape}")    # → (1, 28)
```

• END OF REPORT •

Uditya Narayan Tiwari

Developer & Technical Content Creator

AI/LLM Tooling · Model Context Protocol ·

Python

[Portfolio ↗](#)

[GitHub ↗](#)

[LinkedIn ↗](#)

[Knowledge Base ↗](#)

This report provides a comprehensive technical examination of the Transformer decoder. All mathematical formulations reference the original published literature (Vaswani et al., 2017 and subsequent works). The PyTorch implementation is for educational purposes. © 2026 Uditya Narayan Tiwari.

APPENDIX

Notation Reference

All mathematical symbols used in this report with their tensor shapes and definitions.		
SYMBOL	SHAPE / TYPE	DEFINITION
d_{model}	scalar	Model dimension; 512 in original Transformer, up to 12 288 in GPT-3
d_k, d_v	scalar	Per-head key/value dimension; $d_k = d_{\text{model}}/h$
d_{ff}	scalar	Feed-forward inner dimension; typically $4 \times d_{\text{model}}$
h, N	scalar	h = number of attention heads (8 original); N = number of stacked decoder blocks (6 original)
T, n	scalar	Target sequence length T ; source sequence length n
Q, K, V	$\mathbb{R}^{T \times d_k}$	Query, Key, Value matrices after linear projection
$\mathbf{H}_{\text{enc}}$	$\mathbb{R}^{n \times d_{\text{model}}}$	Encoder final hidden states; serve as K and V in cross-attention
M	$\mathbb{R}^{T \times T}$	Causal mask: upper-triangular $-\infty$, lower-triangular + diagonal are 0
W_Q, W_K, W_V	$\mathbb{R}^{d_{\text{model}} \times d_k}$	Learned projection matrices for queries, keys, values (per head)
W^O	$\mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$	Output projection of multi-head attention; re-projects concatenated heads
W_1, W_2	$\mathbb{R}^{d \times d_{\text{ff}}}$	FFN inner/outer weight matrices; b_1, b_2 are corresponding bias vectors
γ, β	$\mathbb{R}^{d_{\text{model}}}$	Learned scale and shift parameters of Layer Normalisation
$\mathcal{V}$	set	Vocabulary; $ \mathcal{V} $ is vocabulary size (e.g. 50 257 for GPT-2)
$\mathcal{L}, \text{PPL}$	scalar	Cross-entropy training loss; perplexity = $\exp(\mathcal{L})$